

بسم الله الرحمن الرحيم

صلى الله و سلم على سيدنا محمد و على اله و صحبه و سلم تسليما كثيرا الى يوم الدين

Arabic Diacritics Pipeline (Character-Level Classifier)

Stage I: Data Preprocessing:

- A. **Data Cleaning**: removing undesired words, character
Ex: Punctuation/Symbols, HTML Tags, Whitespace, (diacritics removal if existing)
Tools: Python's re (Regex)
- B. **Text normalization**: handles different ways of typing an Arabic character
Ex: Alef Normalization (أ، آ، إ، ؤ), Hamza/Ta' Marbuta (ة), (ه), Tatweel Removal (-)
Tools: PyArabic, camel_tools, Tnkeeh
- C. **Tokenization**:

Tokenization Scheme	Description	Use Case in Diacritization	Recommended Models
Word-Level Tokenization	Splits the sentence based on whitespace and punctuation (e.g., ذهب على إلى الشاطئ \$\\rightarrow\$ [ذهب, على, الشاطئ, إلى]).	Baseline/Word-Level Feature Extraction. Used for BoW, TF-IDF, or as the initial segmentation for Word-level Encoders.	HMM, CRF, or Bi-LSTM models.
Character-Level Tokenization	Treats every single character (including spaces) as a token (e.g., ذ ه ب ع ل ي (...)).	Character-Level Classification. This is the final required step for the Diacritic Classifier.	Character-level RNN/LSTM/Transformer models.
Morphological Tokenization (Segmentation)	Splits words into their constituent morphemes: Prefix(es), Stem, Suffix(es) (e.g., والمصريون و + ال + + مصري + ون).	Advanced Feature. Provides rich linguistic features (e.g., POS tag, clitic info) crucial for predicting case endings (last diacritic) , which is the hardest part of diacritization.	Hybrid models, or as a pre-processing step to generate features for deep learning.
Subword Tokenization (BPE/WordPiece)	Splits text into frequently occurring subword units, balancing the vocabulary size with morphological information.	State-of-the-Art Deep Learning. Standard for Transformer-based models (like AraBERT) to handle the large Arabic vocabulary and OOV words.	AraBERT , Transformer Encoders (High-Performance Path).

These are the 3 most common subword tokenizers used in Arabic NLP projects ([A Comprehensive Analysis of Various Tokenizers for Arabic Large Language Models](#))

Feature	WordPiece (WP)	Byte-Level BPE (BBPE)	SentencePiece (SP)
Merging Strategy (Key Difference)	Maximizes Likelihood: Selects the symbol pair whose merge maximizes the likelihood of the training data.	Maximizes Frequency: Selects the most frequent consecutive token pair to merge.	Supports BPE and Unigram algorithms. The Unigram model uses a probability-based approach and iteratively prunes the vocabulary.
Initial Pre-tokenization	Required: Typically requires the text to be split into 'words' first (usually by whitespace and punctuation).	Required: Separates text into words based on pre-tokenization.	Not Required: Treats the raw input as a sequence of Unicode characters, making it truly language-independent.
Base Vocabulary	Starts with individual characters from the training data.	Uses a fixed 256-byte set instead of characters.	Starts with basic characters from the training set.
Handling of Whitespace	Implicit: Relies on whitespace for initial word split; subwords are often prefixed with ## to denote continuation (e.g., ##comfortable).	Implicit: Similar to BPE, relies on whitespace for initial split.	Explicit: Treats spaces as a normal character (e.g., typically represented by _ or) and includes them in the vocabulary, allowing it to tokenize raw, pre-tokenized text seamlessly.
OOV Handling	Good, but can struggle with OOV words by over-splitting.	Excellent: Since the base vocabulary covers all 256 possible bytes, it guarantees no OOV tokens (every sequence is representable).	Good, especially for languages without clear word boundaries.
Prominent Models	BERT, AraBERT, DistilBERT, Arabic-BERT.	GPT-2, RoBERTa.	T5, ALBERT, XLNet.

Stage II: Feature Extraction:

In this phase we extract the features that will be used to train our classifier model. This means that we convert text into numerical data using different techniques such as embeddings.

Feature Extraction is divided into 3 main categories:

Classical/statistical Features (Required baseline), Static word Embeddings (required for comparison), Contextual Embeddings (State – of – the- art)

A. Classical/statistical Features (Required baseline)

Feature	Description	Rationale for Diacritization
Term Frequency-Inverse Document Frequency (TF-IDF)	A statistical measure that reflects how important a word is to a document in a corpus. It weights common words lower and domain-specific words higher.	Useful for identifying key (less common, thus more informative) words that dictate the sentence structure (e.g., proper nouns, technical terms).
Bag of Words (BoW)	Represents text as the count of words in a vocabulary, ignoring grammar and word order.	A simple, basic benchmark. Good for classification where word order is less critical, but less effective for sequence tasks like diacritization.
N-grams (Word/Character)	Sequences of \$N\$ items (words or characters). Character N-grams can capture morphological patterns, which is highly relevant in Arabic.	Character N-grams are crucial for handling Out-Of-Vocabulary (OOV) words and word roots in Arabic, as FastText does.
Linguistic Features	Morphological characteristics, Part-of-Speech (POS) tags , and grammatical case.	Can be integrated as a feature input to a model (e.g., adding a POS tag dimension to the word

Feature	Description	Rationale for Diacritization
		vector). Highly informative for predicting case endings (l'rāb).

B. Static word Embeddings (required for comparison)

Feature	Description	Rationale for Diacritization
Word2Vec (CBOW / Skip-gram)	A two-layer neural net architecture. CBOW predicts the word from its context; Skip-gram predicts the context from the word.	Captures word similarity (e.g., "teacher" and "instructor" are close in vector space). A strong representation layer for the input of an LSTM.
FastText	An extension of Word2Vec that uses character N-grams (sub-word units) in addition to the word itself.	Highly recommended for Arabic because it can generate vectors for unknown or rare words (OOV) based on their morphology, which is critical for diacritization accuracy.
GloVe (Global Vectors for Word Representation)	Combines global matrix factorization (co-occurrence) methods with local context window methods (like Word2Vec).	Often yields performance competitive with Word2Vec/FastText and is another excellent option for your second required feature.

Feature	Description	Rationale for Diacritization
AraVec	A set of Arabic word embedding models pre-trained on large Arabic corpora from different domains (Tweets, WWW, Wikipedia).	Use a pre-trained Arabic-specific model like AraVec to benefit from large-scale training data without needing to train from scratch.

C. Contextual Embeddings (Required State-of-the-Art)

Feature	Description	Rationale for Diacritization
BERT / mBERT	Bidirectional Encoder Representations from Transformers. Uses a Transformer encoder and is pre-trained on masked language modeling, yielding deep, bidirectional representations.	The gold standard. Directly solves ambiguity by providing a context-aware vector for every word, which is essential for predicting the complex case-endings in Arabic.
AraBERT / CAMElBERT / MARBERT	BERT models specifically pre-trained on large, clean Arabic corpora.	Highly Recommended. Using an Arabic-specific BERT variant (like AraBERT) will significantly outperform a multilingual BERT (mBERT) on this task.

Stage III: Model Building:

We have either machine learning models or deep learning models

A. Machine Learning Models

Model Type	Available Models	Core Concept & Relevance
Generative Models	Hidden Markov Model (HMM)	Models the joint probability $P(X, Y)$ of the input sequence (X , the characters) and the output sequence (Y , the diacritics). HMM makes a strong assumption that the label for a character only depends on the immediately preceding labels (Markov assumption).
Discriminative Models	Conditional Random Field (CRF)	Models the conditional probability $P(Y$
Hybrid	Bi-LSTM + CRF	While the LSTM acts as a deep learning encoder (feature extractor), the CRF layer is placed on top of the LSTM outputs. It learns the constraints and dependencies between the output labels (diacritics), ensuring the predicted sequence of diacritics is grammatically and linguistically valid.

B. Deep Learning Models

Model Type	Available Models	Core Concept & Relevance
Recurrent Neural Networks (RNN)	Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU)	RNNs process sequences step-by-step, maintaining a hidden state (memory) to capture context. <i>LSTMs</i> and <i>GRUs</i> are variants that use gating mechanisms (Input, Forget, Output gates) to solve the vanishing gradient problem, allowing them to capture long-term dependencies in long sentences. Bidirectional (Bi-LSTM/Bi-GRU) models are highly effective, as they read the sentence both forward and backward to capture context from the whole sequence.

Model Type	Available Models	Core Concept & Relevance
Attention-Based Models	Transformer, BERT (e.g., AraBERT)	These models replace recurrence with a Self-Attention mechanism, allowing the model to weigh the importance of all input tokens simultaneously, regardless of their distance. They are highly parallelizable and excel at capturing global context over very long sequences. Transformer-based models, especially those pre-trained on Arabic (like AraBERT), currently achieve state-of-the-art results in sequence labeling tasks.

Action Course:

1) TF-IDF:

- a. **Pros:** Simple and fast + Adds Weights to words reflecting their importance
- b. **Cons:** Neglects Word order and semantic meaning
- c. **Tokenizer:** Word level tokenizer (word-split)
- d. **Model:** HMM or CRF

2) Word Embeddings (Static/ Pretrained):

- a. **Pros:** captures semantic meaning + reduces dimensions
- b. **Cons:** Cannot resolve polysemy (same word having different meanings (e.g., distinguishing between past tense verb ذَهَبَ / *dhahaba* 'he went' and noun ذَهَبٌ / *dhahabun* 'gold' based on the surrounding sentence)
- c. **Tokenizer:** Simple Sub word level tokenizer (FastText or other, based on the feature engineering model)
- d. **Model:** LSTM, Bi-LSTM, GRU, Bi-GRU

3) Contextual Embeddings:

- a. **Pros:** Resolves Polysemy
- b. **Cons:** Complex
- c. **Tokenizer:** Sub word tokenizers (BBPE or WordPiece)
- d. **Model:** Transformer, BERT, AraBERT

Customized Pipeline:

Pipelines will be divided into 2 main categories: Classical/Hybrid Architectures, deep Learning Architectures

Pipeline Name	Architecture Components (Flow)	Core Strength (Why it's used)
HMM-Based	Feature Extractor $\rightarrow$ HMM (Model)	Strong, highly interpretable baseline . Relies on transition and emission probabilities.
MEMM-Based	Feature Extractor $\rightarrow$ Maximum Entropy Markov Model (MEMM)	Discriminative alternative to HMM. Models conditional probability $P(y \mid x)$ but suffers from the Label Bias Problem .
CRF-Only	Feature Extractor $\rightarrow$ CRF (Model)	Captures long-range dependencies and enforces global constraints across the entire label sequence, superior to HMM/MEMM.

Pipeline Name	Architecture Components (Flow)	Core Strength (Why it's used)
Bi-LSTM (Stacked RNN)	Word Embeddings $\rightarrow$ (Stacked) Bi-LSTM/Bi-GRU $\rightarrow$ Softmax/Dense Layer	Captures bidirectional context across the sequence. Good performance with static embeddings (e.g., FastText).
Bi-LSTM-CRF	Word Embeddings $\rightarrow$ Bi-LSTM (Encoder) $\rightarrow$ CRF (Decoder/Inference)	Standard Hybrid Model. Bi-LSTM learns the features and context, and the CRF layer then ensures the predicted diacritic sequence is grammatically and sequentially

Pipeline Name	Architecture Components (Flow)	Core Strength (Why it's used)
		optimal. Highly Recommended for Pipeline 2.
Transformer-Softmax	Contextual Embeddings (BERT/AraBERT) $\rightarrow$ Transformer Encoder $\rightarrow$ Softmax Layer	State-of-the-Art (SOTA). Global context capture via self-attention is superior to recurrent models. The Encoder (e.g., BERT) is often sufficient to predict the labels directly.
Transformer-CRF	Contextual Embeddings $\rightarrow$ Transformer Encoder $\rightarrow$ CRF (Decoder)	Combines SOTA contextual encoding with the sequence-level optimization of CRF. Often used to gain a slight edge over the Softmax approach, particularly useful when there are strong output dependencies (like Arabic diacritics).

Layer	Component/Model	Role & Rationale
Data Preprocessing	Normalization & Tokenization	Clean and segment the text into words ³ .
Word-Level Feature Ext.	Three Required Features (TF-IDF, Embeddings, Contextual Embeddings)	Generate a vector representation for each word. You will train three separate models (one for each feature) within this pipeline and select the best one ⁴ .

Layer	Component/Model	Role & Rationale
Word-Level Encoder	Optional: Bi-LSTM/RNN	<i>Optional, but Recommended.</i> To capture context across the sentence, especially for TF-IDF/static embeddings, which lack inherent context ⁵ .
Character-Level Classifier	Conditional Random Field (CRF) or Hidden Markov Model (HMM)	A classic and robust model for sequence labeling. CRF models the conditional probability $P(\text{Diacritics} \mid \text{Characters})$, making it a strong baseline for character-level classification ⁶⁶ .

Layer	Component/Model	Role & Rationale
Data Preprocessing	Normalization & Tokenization	Same as Pipeline 1 ⁷ .
Word-Level Feature Ext.	Best Feature from Pipeline 1 (e.g., Contextual Embeddings/Trainable Embeddings)	Use the single best feature chosen from the first pipeline as input ⁸ .
Encoder-Decoder Architecture	Transformer (Preferred) or Bi-LSTM	Transformer: Excellent at capturing long-range dependencies and inherent context. It functions as both the Word-Level and Character-Level Encoder. This is the standard for modern Seq2Seq tasks. Bi-LSTM: A required model type ⁹ . It captures features across the sequence (the word's surrounding words and characters) ¹⁰ .

Layer	Component/Model	Role & Rationale
Character-Level Output	Dense Layer + Softmax	The final layer after the Encoder (e.g., Bi-LSTM) outputs a probability distribution over the set of diacritics for each character, framing it as a classification problem ¹¹ .

Layer	Component	Architectural Rationale
Word-Level Feature Ext.	TF-IDF, Static Embeddings (e.g., Word2Vec)	These methods generate a fixed-size vector for each word. They are simple to compute and serve as a direct comparative test for the <i>value</i> of the feature itself. Limitation: They lack global context, which is why the next layer is crucial.
Word-Level Encoder	Bi-LSTM	Why Bi-LSTM? An LSTM (Long Short-Term Memory) is designed to handle sequential data and capture long-term dependencies. A Bi-directional LSTM reads the word features in both the forward and backward directions of the sentence (e.g., from right-to-left and left-to-right in Arabic). This is essential for understanding the syntactic context (e.g., subject-verb agreement) required for correct case-endings (l'rāb).
Character-Level Classifier	CRF or HMM	Why CRF/HMM? These are probabilistic models for sequence labeling. The CRF (Conditional Random Field) is preferred over a simple Softmax layer because it explicitly models the <i>dependencies between output labels</i> . For diacritization, this means if we predict a <i>Shadda</i> ($\text{\text{}}\text{\text{}}\text{\text{}}$) on a character, the CRF layer can enforce a rule that the subsequent

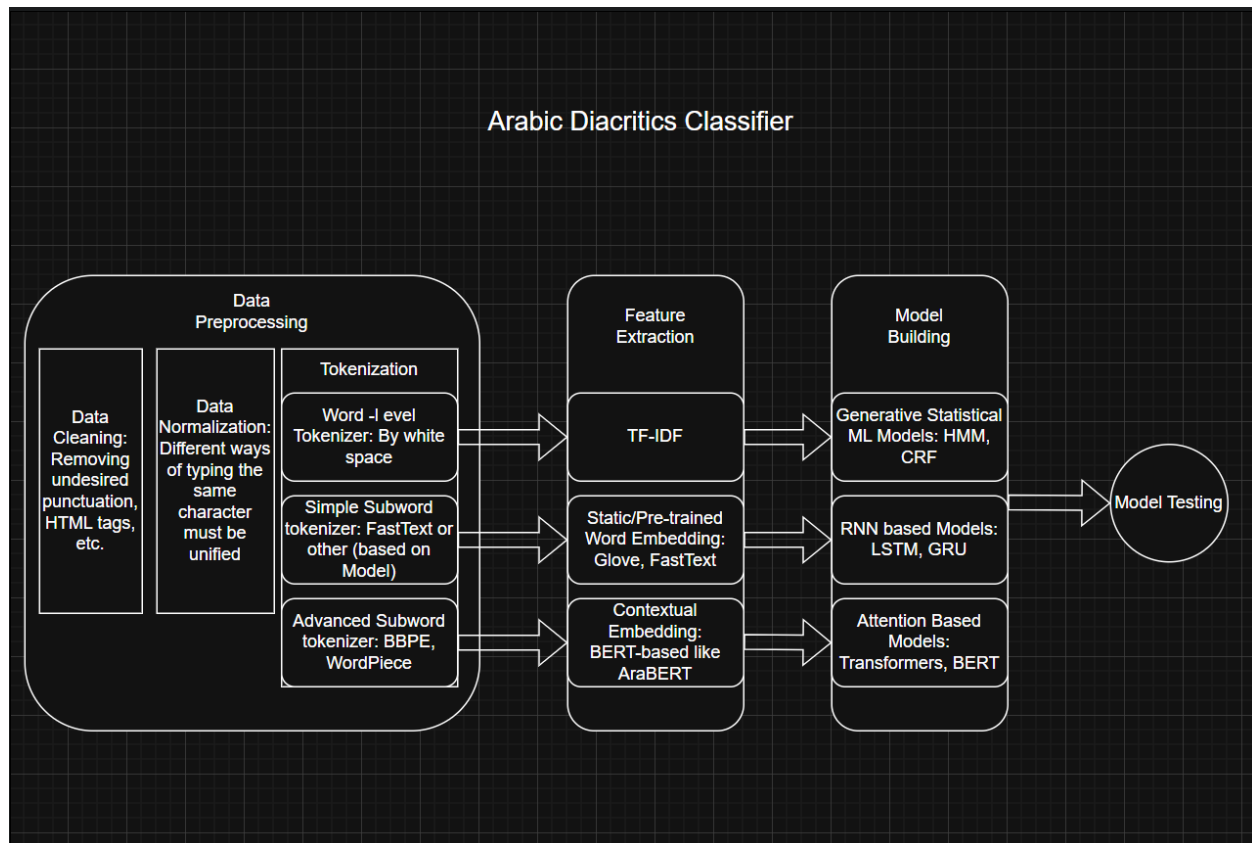
Layer	Component	Architectural Rationale
		prediction cannot be a $\text{\text{Sukūn}}$ ($\text{\text{}}$), leading to more grammatically sound sequences.

Layer	Component	Architectural Rationale
Input Feature	Contextual Embeddings (e.g., AraBERT/mBERT)	Why Contextual Embeddings? Models like BERT are pre-trained on massive Arabic corpora to provide a feature vector for a word that is unique <i>to its specific sentence context</i> . This eliminates much of the initial ambiguity present in Arabic. You can use these features in a feature-based approach or fine-tune the entire model.
Encoder-Decoder Architecture	Transformer or Stacked Bi-LSTMs	Why Transformer? The Transformer's Attention Mechanism allows it to simultaneously weigh the relevance of <i>every</i> other character/word in the sentence when deciding the diacritic for the current character. This offers superior capture of long-range dependencies compared to RNNs, which process data sequentially. This is crucial for resolving syntactic ambiguity.
Character-Level Output	Dense Layer + Softmax	Why Softmax? When using a Transformer or a robust Stacked Bi-LSTM, the upstream layers have already encoded the complex context. A simple Softmax layer on top treats the problem as an N-way classification (where N is the number of possible diacritic combinations) for each character. A CRF can still be added on top

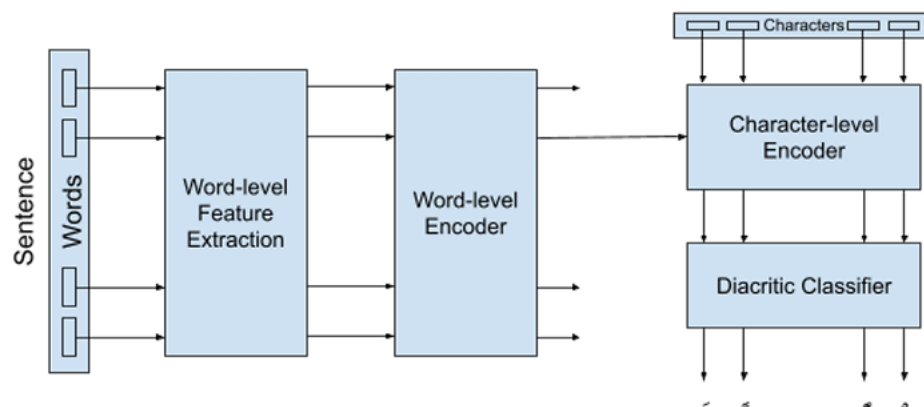
Layer	Component	Architectural Rationale
		of a Transformer, but often the complexity is not warranted if the attention mechanism is powerful enough.

How to design your pipeline?

Design Question	Implication for Arabic Diacritization	Recommended Layers
1. What context is needed?	Diacritics require both preceding and following context within the word and the sentence.	Bi-directional Architectures (Bi-LSTM, Transformer Encoder)
2. How to handle long sequences?	Sentences can be long, and dependencies (like case endings) can span many words.	LSTM (handles vanishing gradient), Transformer (excels at long-range attention).
3. Does the output depend on the previous output?	Yes, the diacritics form a grammatically valid sequence. $\text{\text{Fatha}}$ and $\text{\text{Shadda}}$ can combine, but not $\text{\text{Sukūn}}$ and $\text{\text{Shadda}}$.	CRF (Conditional Random Fields) layer on top of the encoder.
4. How complex/accurate must the system be?	The competition uses DER (Diacritic Error Rate). Max accuracy requires the best models.	Pipeline 2 (Transformer/Bi-LSTM Stack) is your highest-ceiling option.



Document Pipeline:



What is the difference?

Your Custom Pipeline	Project Document Pipeline
Abstraction: High-level overview, grouping processes by <i>methodology</i> (Statistical vs. Deep Learning).	Abstraction: Low-level architectural flow, detailing the <i>internal steps</i> of a sequence model.
Stage Mapping: "Model Building" block.	Stage Mapping: "Word-level Encoder," "Character-level Encoder," and "Diacritic Classifier."

- 1) **Word level Encoder:** Extract features from current word, previous and next words
- 2) **Character level Encoder:** Extract features from current character, previous and next characters

In the deep learning models, these encoders are implemented by the recurrent layers or attention mechanisms. For the classical statistical models, they work differently from deep learning encoders, they rely on explicit features (counts and probabilities) rather than hierarchal structures.